

Reviewer Pack — Minimal Rank of Quantum Entanglement Matrix Bounds Circuit W...

Entry f674f8cfc798 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 13:07:49 UTC

Minimal Rank of Quantum Entanglement Matrix Bounds Circuit Weights of Boolean Functions

Entry ID: f674f8cfc798 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 13:07:49 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Entanglement) **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

{‘p1’: ‘For any given n -bit boolean function f , the minimal rank of the density matrix ρ representing a maximally entangled state in the support of f is $\Theta(\log n)$.’, ‘p2’: ‘Furthermore, for all boolean functions, the circuit weight $W(f)$ satisfies $W(f) \leq 2^{\rho(f)}$.’, ‘p3’: ‘If there exists a boolean function f such that $\rho(f) > \log n$ and $W(f) < 2^{\rho(f)}$, then $P = NP$.’}

Rationale (proposer’s reasoning):

{‘p1’: ‘Quantum entanglement is a fundamental resource in quantum information theory, and its properties have been extensively studied. The minimal rank of the entanglement matrix could potentially provide insights into the complexity of boolean functions.’, ‘p2’: ‘If the conjecture holds, it would suggest a deep connection between quantum mechanics and computational complexity theory, providing new ways to study circuit complexity.’, ‘p3’: ‘The quantitative relationship between entanglement rank and circuit weight offers a novel approach for testing P vs NP problems.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7531f24e6729cebd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all generated boolean functions, the minimal rank of the entanglement matrix is within a factor of 3 from $\log n$ and the circuit weight does not exceed 2 raised to the power of the minimal rank.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum entanglement" AND "Boolean function complexity" - "density matrix rank" AND "circuit weight" - "maximally entangled state" AND "NP completeness"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=30.9s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def density_matrix(f):
        n = len(f)
        rho = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if f[i] == f[j]:
                    rho[i][j] += 1
        return rho

    def min_rank(matrix):
        n = len(matrix)
        rank = 0
        for i in range(n):
            if matrix[i][i] != 0:
                rank += 1
                for j in range(i + 1, n):
                    matrix[j][i] /= matrix[i][i]
                for j in range(n):
                    if j != i:
                        for k in range(i + 1, n):
                            matrix[j][k] -= matrix[j][i] * matrix[i][k]
        return rank
```

```

def circuit_weight(f):
    # Placeholder function for actual circuit weight calculation
    return len(f)

n = random.randint(5, 30)
f = generate_boolean_function(n)
rho = density_matrix(f)
rank = min_rank(rho)
W_f = circuit_weight(f)

log_n = math.log2(n)
if rank < log_n * (1/3) or rank > log_n * (4/3):
    return {
        "metric_name": "min_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Rank {rank} is not within a factor of 3 from log n = {log_n}"
    }
if W_f > 2 ** rank:
    return {
        "metric_name": "circuit_weight",
        "metric_value": W_f,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Circuit weight {W_f} exceeds 2^rank = {2 ** rank}"
    }

return {
    "metric_name": "min_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if r["counterexample"])

```

```

print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
else:
print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify if the pre-registered support conditions were met. | next: Re-run the test to ensure it completes successfully and meets the pre-registered criteria for support.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12064
2	propose	ollama_remote	glm4:latest	0	0	15266
3	propose	ollama_remote	glm4:latest	0	0	13722
4	propose	ollama_remote	glm4:latest	0	0	12583
5	preregistration	ollama_remote	glm4:latest	0	0	5608
6	novelty	ollama_remote	glm4:latest	0	0	4538
7	novelty	ollama_remote	glm4:latest	0	0	5312
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41343
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10934
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	48930
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11405
12	judge	ollama_remote	glm4:latest	0	0	61926

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 243633 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in <research/audit/f674f8cfc798.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f674f8cfc798.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f674f8cfc798.tar.gz` (if generated)